

■ 강의명: CSCI89

■ 주차: Lecture 12

■ 교수명: UIUC 교수진

■ 목적: CSCI89 Lecture 12는 인공지능 분야의 핵심 기술인 어텐션 메커니즘(Attention Mechanism)과 트랜스포머(Transformer)를 다룹니다. 이 강의는 순환 신경망(RNN)의 한계(예: 장거리 의존성 학습의 어려움, 병렬 처리의 비효율성)를 극복하기 위해 어텐션 메커니즘이 어떻게 도입되었는지 설명합니다. 궁극적으로 트랜스포머는 RNN 부분을 완전히 제거하고 오직 어텐션 메커니즘만으로 시퀀스 모델링을 수행하여, 병렬 처리와 장거리 의존성 학습을 극대화합니다.

Contents

1	용어 정리	2
2	퀴즈 복습	3
2.1	조건부 랜덤 필드(CRF) vs. 은닉 마르코프 모델(HMM)	3
2.2	LSTM과 CRF 결합의 이점	3
2.3	일반 오토인코더의 잠재 공간	4
2.4	변이형 오토인코더(VAE)의 평균(μ)	4
2.5	변이형 오토인코더(VAE)의 출력 결정론 여부	4
3	과제 마감일 안내	5
4	어텐션 메커니즘 (Attention Mechanism)	6
4.1	순환 신경망(RNN)의 한계	6
4.2	어텐션 메커니즘의 등장 배경	6
4.3	어텐션 메커니즘의 작동 원리	7
4.4	어텐션 메커니즘의 장점	8
5	트랜스포머 (Transformers)	9
5.1	어텐션 기반 RNN의 한계	9
5.2	트랜스포머 아키텍처 개요	9
5.3	트랜스포머의 핵심 구성 요소	11
5.3.1	임베딩 및 위치 인코딩 (Embeddings and Positional Encoding)	11
5.3.2	멀티 헤드 어텐션 (Multi-Head Attention)	11
5.3.3	잔차 연결 및 정규화 (Residual Connections and Normalization)	13
5.3.4	마스크드 멀티 헤드 어텐션 (Masked Multi-Head Attention)	13
5.4	트랜스포머의 활용: BERT와 GPT	14
6	트랜스포머의 장점	15
7	빠르게 훑어보기 (1 페이지 요약)	16
8	자주 묻는 질문 (FAQ)	17
9	체크리스트	18

1 용어 정리

다음 표는 강의에서 다루는 주요 용어들을 비전공자도 이해하기 쉽게 설명합니다.

용어	쉬운 설명	원어	비고
조건부 랜덤 필드	문맥을 고려하여 시퀀스의 각 요소에 라벨을 붙이는 모델	Conditional Random Field (CRF)	HMM보다 복잡한 의존성 모델링
은닉 마르코프 모델	숨겨진 상태를 통해 관찰된 시퀀스를 설명하는 통계 모델	Hidden Markov Model (HMM)	현재 상태가 이전 상태에만 의존
장단기 기억망	장기 의존성 문제를 해결하기 위해 고안된 특별한 RNN	Long Short-Term Memory (LSTM)	기억 셀을 통해 정보 유지
오토인코더	입력 데이터를 압축(인코딩)하고 다시 복원(디코딩)하는 신경망	Autoencoder	데이터 압축 및 특징 학습
변이형 오토인코더	잠재 공간에 확률 분포를 도입하여 생성 능력을 강화한 오토인코더	Variational Autoencoder (VAE)	노이즈를 추가하여 비결정론적 출력
어텐션 메커니즘	입력 시퀀스의 특정 부분에 더 집중하여 정보를 처리하는 방식	Attention Mechanism	RNN의 한계 극복, 문맥 벡터 사용
트랜스포머	RNN 없이 어텐션 메커니즘만으로 시퀀스를 처리하는 모델	Transformer	병렬 처리 및 장거리 의존성 학습에 탁월
위치 인코딩	시퀀스 내 토큰의 순서 정보를 모델에 알려주는 방법	Positional Encoding	어텐션의 순열 불변성 문제 해결
쿼리 (Q)	현재 처리하려는 토큰의 '질문' 또는 '초점'	Query	어떤 정보를 찾고 있는지 나타냄
키 (K)	시퀀스 내 다른 토큰의 '키' 또는 '식별자'	Key	쿼리와 얼마나 관련이 있는지 비교
값 (V)	시퀀스 내 다른 토큰의 '값' 또는 '정보 내용'	Value	쿼리와의 관련성에 따라 추출될 실제 정보
멀티 헤드 어텐션	여러 개의 어텐션 메커니즘을 병렬로 사용하여 다양한 관점의 정보 포착	Multi-Head Attention	다양한 관계를 동시에 학습
스케일드 닷-프로덕트 어텐션	쿼리와 키의 내적을 스케일링하여 어텐션 점수를 계산하는 방식	Scaled Dot-Product Attention	학습 안정화
잔차 연결	신경망의 특정 레이어 입력을 출력에 직접 더하는 연결 방식	Residual Connection (Shortcut)	깊은 네트워크 학습 용이
레이어 정규화	각 샘플의 모든 특징(피치)에 대해 정규화를 수행하는 기법	Layer Normalization	학습 안정화
마스크드 어텐션	디코더에서 미래 토큰을 보지 못하도록 가리는 어텐션 방식	Masked Attention	텍스트 생성 시 미래 정보 유출 방지
BERT	트랜스포머의 인코더 부분을 활용한 양방향 문맥 이해 모델	Bidirectional Encoder Representations from Transformers	텍스트 이해 작업에 특화
GPT	트랜스포머의 디코더 부분을 활용한 텍스트 생성 모델	Generative Pre-trained Transformer	텍스트 생성 작업에 특화

2 퀴즈 복습

이번 강의는 지난 퀴즈에 대한 복습으로 시작되었습니다. 각 질문과 그에 대한 정답 및 설명을 자세히 살펴보겠습니다.

2.1 조건부 랜덤 필드(CRF) vs. 은닉 마르코프 모델(HMM)

질문 1: CRF의 장점

은닉 마르코프 모델(Hidden Markov Models, HMM)과 비교했을 때 조건부 랜덤 필드(Conditional Random Fields, CRF)를 사용하는 것의 장점은 무엇인가요?

정답 및 설명

정답: CRF는 HMM보다 더 복잡한 의존성을 모델링할 수 있습니다.

- HMM은 현재 상태가 오직 이전 상태에만 의존한다고 가정하는 반면, CRF는 미래 토큰과의 결합 분포를 포함하여 더 넓은 문맥과 복잡한 의존성을 고려할 수 있습니다.
- 이는 특히 자연어 처리와 같이 단어의 의미가 과거뿐만 아니라 미래 단어에도 영향을 받는 경우에 매우 중요합니다.

오해 방지: CRF의 장점은 입력 변수가 이산적인지 연속적인지 여부와는 관련이 없습니다. 두 모델 모두 이산 또는 연속 변수에 사용될 수 있으며, 핵심은 시간 흐름과 문맥 의존성을 모델링하는 방식에 있습니다. HMM은 엄격한 독립성 가정을 가지지만, CRF는 이러한 가정을 완화하여 더 현실적인 언어 모델링이 가능합니다.

2.2 LSTM과 CRF 결합의 이점

질문 2: LSTM과 CRF 결합의 이점

LSTM(Long Short-Term Memory)과 조건부 랜덤 필드(CRF)를 결합하는 것의 주요 이점은 무엇인가요?

정답 및 설명

정답: 모델이 과거 및 미래 문맥을 모두 포착하는 능력을 향상시킵니다.

- LSTM은 시퀀스 데이터에서 장기 의존성을 학습하는 데 탁월하며, 과거 문맥을 효과적으로 포착합니다.
- CRF는 시퀀스 라벨링 작업에서 출력 라벨 간의 의존성을 모델링하여, 전체 시퀀스에 걸쳐 일관된 예측을 할 수 있도록 돕습니다.
- 이 둘을 결합하면 LSTM이 추출한 강력한 특징 표현에 CRF가 시퀀스 전체의 문맥적 제약을 추가하여, 과거와 미래의 문맥을 동시에 고려한 더욱 정확한 예측이 가능해집니다.

2.3 일반 오토인코더의 잠재 공간

질문 3: 일반 오토인코더의 잠재 공간

변이형 오토인코더(Variational Autoencoder)가 아닌 일반 오토인코더(Regular Autoencoder)를 사용하여 셰익스피어의 텍스트를 인코딩한 후, 인코딩된 표현을 수동으로 조금씩 이동시키면 출력에 어떤 변화가 생길까요?

정답 및 설명

정답: 일반 오토인코더의 인코딩 공간(잠재 공간)은 구조화되어 있지 않으므로, 인코딩을 조금만 이동시켜도 출력이 완전히 파괴되거나 의미 없는 텍스트가 될 수 있습니다.

- 일반 오토인코더는 단순히 입력 데이터를 압축하고 복원하는 데 최적화되어 있습니다. 이 과정에서 생성되는 잠재 공간은 연속적이고 의미 있는 구조를 가지도록 강제되지 않습니다.
- 따라서 잠재 공간에서 작은 변화를 주더라도, 디코더는 이를 의미 있는 데이터로 해석하지 못하고 완전히 다른(또는 무의미한) 출력을 생성할 가능성이 높습니다. 이는 마치 이미지에서 픽셀 값을 조금만 바꿔도 이미지가 깨지는 것과 유사합니다.

2.4 변이형 오토인코더(VAE)의 평균(μ)

질문 4: VAE의 평균(μ)은 입력의 함수인가?

변이형 오토인코더(VAE)를 훈련할 때, 평균(μ)은 입력의 함수인가요?

정답 및 설명

정답: 네, μ 는 입력의 결정론적 함수입니다.

- VAE에서 인코더는 입력 데이터를 받아 잠재 공간의 평균(μ)과 분산(σ)을 출력합니다.
- 이 μ 는 입력에 따라 결정되는 값으로, 동일한 입력이 주어지면 항상 동일한 μ 를 생성합니다. 즉, μ 를 계산하는 과정은 일반적인 신경망과 같이 결정론적입니다.
- 하지만 VAE의 최종 잠재 벡터는 이 μ 와 σ 로부터 샘플링된 노이즈를 더하여 생성되므로, 전체 출력 과정은 비결정론적입니다.

2.5 변이형 오토인코더(VAE)의 출력 결정론 여부

질문 5: VAE의 출력 결정론 여부

변이형 오토인코더(VAE)에서 특정 입력이 주어졌을 때, 네트워크의 출력은 결정론적인가요?

정답 및 설명

정답: 아닙니다. VAE의 출력은 결정론적이지 않습니다.

- VAE는 잠재 공간에서 샘플링 과정에 노이즈(noise)를 추가합니다.
- 이는 동일한 입력이 주어지더라도, 매번 다른 노이즈가 샘플링될 수 있으므로, 네트워크의 최종 출력이 달라질 수 있음을 의미합니다.
- 이러한 비결정론적 특성은 VAE가 새로운 데이터를 생성하는 능력의 핵심입니다.

3 과제 마감일 안내

강의 중 학생의 질문에 따라 과제 마감일이 조정될 예정입니다.

과제 10 마감일 조정

- 기존 마감일: 11월 23일 (추정)
- 조정 예정 마감일: 12월 7일 (강의 계획서와 동기화)
- 강사는 강의 후 마감일을 변경할 것이라고 언급했습니다.

4 어텐션 메커니즘 (Attention Mechanism)

어텐션 메커니즘은 순환 신경망(RNN)의 고질적인 한계를 극복하기 위해 도입된 혁신적인 기술입니다. 이 섹션에서는 RNN의 한계부터 어텐션의 작동 원리 및 장점까지 자세히 다룹니다.

4.1 순환 신경망(RNN)의 한계

RNN은 시퀀스 데이터를 처리하는 데 유용하지만, 다음과 같은 여러 가지 근본적인 한계를 가지고 있습니다.

RNN의 주요 한계점

- **장거리 의존성 학습의 어려움 (Vanishing/Exploding Gradients):**
 - RNN은 시퀀스가 길어질수록 먼 과거의 정보가 현재 시점의 예측에 미치는 영향이 줄어들거나 (기울기 소실, Vanishing Gradients), 반대로 너무 커지는 (기울기 폭주, Exploding Gradients) 문제가 발생합니다.
 - LSTM이나 GRU와 같은 변형 모델이 이를 완화했지만, 완전히 해결하지는 못했습니다. 이는 모델이 시퀀스 내의 멀리 떨어진 요소 간의 관계(장거리 의존성)를 학습하기 어렵게 만듭니다.
- **병렬 처리의 비효율성:**
 - RNN은 본질적으로 순차적인 구조를 가집니다. 즉, 다음 시점의 출력을 계산하기 위해 이전 시점의 출력이 필요합니다.
 - 이로 인해 GPU와 같은 병렬 컴퓨팅 장치를 효율적으로 활용하기 어려워, 대규모 데이터셋이나 긴 시퀀스에 대한 학습 시간이 매우 길어집니다.
- **고정 크기 표현 병목 (Fixed-Size Representation Bottleneck):**
 - 인코더-디코더(Encoder-Decoder) 구조의 RNN(예: 시퀀스-투-시퀀스 모델)에서 인코더는 전체 입력 시퀀스를 하나의 고정된 크기의 문맥 벡터(bottleneck vector)로 압축합니다.
 - 이 벡터는 시퀀스의 모든 정보를 담아야 하지만, 길이가 긴 시퀀스의 경우 모든 정보를 효과적으로 압축하기 어렵습니다. 이는 정보 손실로 이어질 수 있습니다.
 - 또한, 문맥에 따라 필요한 정보의 양이 다를 수 있는데, 고정된 크기의 벡터는 이러한 유연성을 제공하지 못합니다.
- **미래 문맥 고려의 어려움:**
 - 기본적인 RNN은 현재 시점의 출력이 과거 정보에만 의존한다고 가정합니다.
 - 하지만 기계 번역과 같은 작업에서는 문장의 의미를 정확히 파악하기 위해 현재 단어뿐만 아니라 미래 단어의 문맥도 중요합니다. RNN은 이러한 미래 문맥을 직접적으로 고려하기 어렵습니다.

4.2 어텐션 메커니즘의 등장 배경

RNN의 이러한 한계, 특히 고정 크기 문맥 벡터의 병목 현상과 장거리 의존성 학습의 어려움을 해결하기 위해 어텐션 메커니즘(Attention Mechanism)이 2014-2015년에 도입되었습니다. 어텐션의 핵심 아이디어는 다음과 같습니다.

어텐션 메커니즘의 핵심 아이디어: 동적 문맥 표현

고정된 하나의 문맥 벡터로 전체 시퀀스를 표현하는 대신, 각 출력 시점마다 입력 시퀀스의 모든 부분에 동적으로 '주의(attention)'를 기울여 새로운 문맥 벡터를 생성합니다. 이 문맥 벡터는 현재 시점의 예측에 가장 관련성이 높은 입력 정보를 가중합하여 반영합니다.

4.3 어텐션 메커니즘의 작동 원리

어텐션 메커니즘은 주로 인코더-디코더 구조에서 디코더가 출력을 생성할 때 인코더의 모든 은닉 상태를 활용하는 방식으로 작동합니다.

1. 인코더의 은닉 상태 (H):

- 입력 시퀀스 $X = (x_1, x_2, \dots, x_T)$ 가 인코더(일반적으로 양방향 RNN 또는 LSTM)를 통과하여 각 시점 j 에 해당하는 은닉 상태 H_j 를 생성합니다.
- 양방향 RNN의 경우, H_j 는 순방향 RNN의 은닉 상태와 역방향 RNN의 은닉 상태를 연결(concatenate)한 벡터가 됩니다. 이는 과거와 미래의 문맥을 모두 담고 있습니다.

2. 정렬 점수 (e):

- 디코더가 현재 시점 t 에서 출력을 예측하려고 할 때, 이전 디코더의 은닉 상태 S_{t-1} 과 인코더의 각 은닉 상태 H_j 가 얼마나 관련이 있는지를 나타내는 정렬 점수(Alignment Score) e_{tj} .
- 이 점수는 $a(S_{t-1}, H_j)$ 와 같은 함수 a 를 통해 계산되며, a 는 일반적으로 피드포워드 신경망(예: $\tanh$ 활성화 함수를 사용하는 단일 레이어)으로 구현됩니다.
- 예시:** $e_{tj} = V^T \tanh(W_s S_{t-1} + W_h H_j)$ (V ,

3. 어텐션 가중치 (alpha):

- 계산된 정렬 점수 e_{tj} (softmax) 어텐션 가중치(Attention Weights)
- $\alpha_{tj} = \text{softmax}(e_{tj})$
- 이 가중치들은 모든 j 에 대해 합이 1이 되는 확률 분포를 형성하며, H_j 가 현재 시점 t 의 출력에 얼마나 '주의'를 기울여야 하는지를 나타냅니다.
- 중요:** α 는 학습 가능한 파라미터가 아니라, S_{t-1} 과 H_j 에 기반하여 동적으로 계산되는 값입니다.

4. 문맥 벡터 (CT):

- 어텐션 가중치 α_{tj} H_j 문맥 벡터(Context Vector) CT .
- $CT = \sum_{j=1}^T (\alpha_{tj} * H_j)$
- CT 는 현재 시점 t 의 출력을 생성하는 데 필요한 가장 관련성 높은 정보를 압축한 동적인 표현입니다.

5. 디코더의 출력 (Yt) 및 다음 은닉 상태 (St):

- 디코더는 이전 은닉 상태 S_{t-1} , 이전 출력 Y_{t-1} , 그리고 새로 생성된 문맥 벡터 CT 를 입력으로 받아 현재 시점의 출력 Y_t 와 다음 은닉 상태 S_t 를 생성합니다.
- $S_t = f(S_{t-1}, Y_{t-1}, CT)$ (여기서 f 는 RNN 셀).

예시: 기계 번역에서의 어텐션 시각화

강의에서는 기계 번역(Neural Machine Translation) 예시를 통해 어텐션 메커니즘의 작동 방식을 시각화했습니다.

- 입력 문장 (영어):** "The agreement on the European Economic Area was signed in 1992."

- **출력 문장 (프랑스어):** "L'accord sur la zone économique européenne a été signé en 1992." 어텐션 가중치(alpha 값)를 시각화한 행렬을 보면, 다음과 같은 특징을 발견할 수 있습니다.
- **단어 정렬:** 영어 단어 "Area" (7번째 단어)가 프랑스어 "zone" (5번째 단어)과 강하게 연결되어 높은 alpha 값을 가집니다. 이는 어텐션 메커니즘이 단어의 순서가 달라도 의미적 연관성을 찾아낼 수 있음을 보여줍니다.
- **자기 참조:** "1992"와 "1992"처럼 동일한 단어는 높은 alpha 값을 가집니다.
- **문맥적 연관성:** "European"과 "européenne"처럼 직접적인 번역 관계에 있는 단어들도 높은 alpha 값을 가집니다.
- **비관련성:** "European"과 "zone"처럼 직접적인 관계가 없는 단어들 사이의 alpha 값은 낮게 나타납니다.

이러한 시각화는 어텐션 메커니즘이 단순히 순차적인 정보 흐름을 따르는 것이 아니라, 입력 시퀀스 전체에서 가장 관련성 높은 정보를 동적으로 찾아내어 번역 품질을 향상시키는 방식을 직관적으로 보여줍니다.

4.4 어텐션 메커니즘의 장점

어텐션 메커니즘은 RNN의 한계를 극복하고 시퀀스 모델링에 여러 가지 중요한 이점을 제공합니다.

어텐션 메커니즘의 주요 장점

- **동적 문맥 집중:** 고정된 문맥 벡터 대신, 각 출력 시점마다 입력 시퀀스의 가장 관련성 높은 부분에 동적으로 '주의'를 기울여 문맥 벡터를 생성합니다. 이는 모델이 필요한 정보에만 집중할 수 있게 합니다.
- **장거리 의존성 포착 능력 향상:** 입력 시퀀스의 모든 은닉 상태를 직접 참조할 수 있으므로, 먼 거리에 있는 토큰 간의 의존성도 효과적으로 학습할 수 있습니다. 이는 RNN의 기울기 소실 문제를 완화합니다.
- **가변 길이 시퀀스 처리:** 고정된 크기의 병목 벡터에 의존하지 않으므로, 다양한 길이의 입력 시퀀스에 유연하게 적용할 수 있습니다. alpha 가중치는 입력 길이에 따라 동적으로 계산됩니다.
- **학습 안정성 및 성능 향상:** RNN의 고질적인 학습 문제를 완화하고, 특히 기계 번역과 같은 복잡한 시퀀스-투-시퀀스 작업에서 모델의 성능을 크게 향상시킵니다.
- **해석 가능성 (Interpretability):** 어텐션 가중치(alpha)를 시각화하여 모델이 어떤 입력 부분에 집중하는지 파악할 수 있어, 모델의 의사결정 과정을 이해하는 데 도움을 줍니다.

5 트랜스포머 (Transformers)

어텐션 메커니즘은 RNN의 성능을 크게 향상시켰지만, 여전히 RNN의 본질적인 한계(병렬화 어려움, 기울기 소실)를 완전히 해결하지 못했습니다. 이러한 한계를 극복하기 위해 등장한 것이 바로 **트랜스포머 (Transformer)**입니다.

5.1 어텐션 기반 RNN의 한계

어텐션 메커니즘이 도입된 RNN 모델(RNN with Attention)은 다음과 같은 한계를 여전히 가지고 있습니다.

어텐션 기반 RNN의 한계점

- **병렬 처리의 어려움:** 어텐션 메커니즘이 추가되었음에도 불구하고, 여전히 기반이 되는 RNN 레이어는 순차적인 계산을 요구합니다. 이로 인해 학습 시간이 길고, 대규모 데이터셋에 대한 병렬 처리가 비효율적입니다.
- **기울기 소실 문제:** RNN 부분은 여전히 기울기 소실 문제를 완전히 벗어나지 못하여, 매우 긴 시퀀스에서는 장거리 의존성 학습에 어려움을 겪을 수 있습니다.
- **지식 전달(Transfer Knowledge)의 어려움:** 특정 작업에 훈련된 모델의 지식을 다른 작업으로 효과적으로 전달하는 것이 여전히 쉽지 않았습니다.

5.2 트랜스포머 아키텍처 개요

2017년 Google Brain 팀이 발표한 논문 "Attention Is All You Need"에서 제안된 트랜스포머는 RNN을 완전히 제거하고 오직 어텐션 메커니즘만으로 시퀀스 데이터를 처리하는 혁신적인 모델입니다.

트랜스포머의 핵심 철학: "Attention Is All You Need"

트랜스포머는 순환(recurrent) 또는 합성곱(convolutional) 레이어 없이, 오직 **어텐션 메커니즘**만으로 입력 시퀀스의 모든 토큰 간의 관계를 직접 모델링합니다. 이를 통해 병렬 처리 능력을 극대화하고 장거리 의존성 학습을 효율적으로 수행합니다.

트랜스포머는 기본적으로 **인코더-디코더(Encoder-Decoder)** 구조를 가집니다.

- **인코더(Encoder):** 입력 시퀀스를 받아 문맥 정보를 담은 연속적인 표현(벡터 시퀀스)으로 변환합니다.
- **디코더(Decoder):** 인코더의 출력을 받아 목표 시퀀스를 생성합니다.

Figure 1: 트랜스포머 아키텍처 개요 (개념도)

트랜스포머 아키텍처의 주요 흐름 (개념도 설명)

인코더 (Encoder):

1. **입력 임베딩 (Input Embeddings):** 입력 토큰(단어 또는 서브워드)을 벡터로 변환합니다.
2. **위치 인코딩 (Positional Encoding):** 임베딩에 토큰의 순서 정보를 추가합니다.
3. **멀티 헤드 어텐션 (Multi-Head Attention):** 입력 시퀀스 내의 모든 토큰 간의 관계를 학습합니다.
4. **잔차 연결 (Add) 및 정규화 (Norm):** 어텐션 레이어의 입력과 출력을 더한 후 레이어 정규화를 수행합니다.
5. **피드포워드 네트워크 (Feed Forward):** 각 위치에 독립적으로 적용되는 두 개의 선형 변환 레이어입니다.
6. **잔차 연결 (Add) 및 정규화 (Norm):** 피드포워드 네트워크의 입력과 출력을 더한 후 레이어 정규화를 수행합니다.
7. 이 과정이 여러 번 반복되어 인코더의 최종 출력을 생성합니다.

디코더 (Decoder):

1. **출력 임베딩 (Output Embeddings):** 생성될 목표 시퀀스의 토큰을 벡터로 변환합니다. (일반적으로 예측을 위해 한 토큰 오른쪽으로 쉬프트된 (shifted right) 입력이 사용됩니다.)
2. **위치 인코딩 (Positional Encoding):** 출력 임베딩에 순서 정보를 추가합니다.
3. **마스크드 멀티 헤드 어텐션 (Masked Multi-Head Attention):** 디코더가 현재 예측하려는 토큰의 미래 정보를 보지 못하도록 마스킹된 어텐션을 수행합니다.
4. **잔차 연결 (Add) 및 정규화 (Norm):** 마스크드 어텐션 레이어의 입력과 출력을 더한 후 레이어 정규화를 수행합니다.
5. **멀티 헤드 어텐션 (Multi-Head Attention):** 인코더의 출력 (Key, Value)과 디코더의 마스크드 어텐션 출력 (Query) 간의 교차 어텐션 (cross-attention)을 수행하여 인코더의 문맥 정보를 활용합니다.
6. **잔차 연결 (Add) 및 정규화 (Norm):** 교차 어텐션 레이어의 입력과 출력을 더한 후 레이어 정규화를 수행합니다.
7. **피드포워드 네트워크 (Feed Forward):** 각 위치에 독립적으로 적용되는 선형 변환 레이어입니다.
8. **잔차 연결 (Add) 및 정규화 (Norm):** 피드포워드 네트워크의 입력과 출력을 더한 후 레이어 정규화를 수행합니다.
9. **선형 (Linear) 및 소프트맥스 (Softmax):** 최종 출력을 다음 토큰의 확률 분포로 변환합니다.

트랜스포머 학습 비용

대규모 트랜스포머 모델(예: GPT-3)을 처음부터 훈련하는 데는 막대한 컴퓨팅 자원이 필요합니다. 강의에서는 이러한 모델의 훈련 비용이 약 460만 달러에서 500만 달러에 달할 수 있다고 언급했습니다. 이는 모델의 복잡성과 학습 데이터의 규모를 반영합니다.

5.3 트랜스포머의 핵심 구성 요소

트랜스포머는 몇 가지 혁신적인 구성 요소로 이루어져 있으며, 이들이 결합하여 강력한 성능을 발휘합니다.

5.3.1 임베딩 및 위치 인코딩 (Embeddings and Positional Encoding)

문제점: 어텐션의 순서 정보 부족

어텐션 메커니즘은 입력 시퀀스의 모든 토큰 간의 관계를 동시에 계산합니다. 이는 병렬 처리에 유리하지만, 토큰의 순서 정보(위치 정보)를 직접적으로 고려하지 못하는 문제가 있습니다. 즉, "고양이가 쥐를 잡았다"와 "쥐가 고양이를 잡았다"를 어텐션만으로는 구분하기 어렵습니다.

해결책: 위치 인코딩 (Positional Encoding)

트랜스포머는 이 문제를 해결하기 위해 위치 인코딩(Positional Encoding)을 도입합니다.

- 각 토큰의 임베딩 벡터에 해당 토큰의 위치 정보를 담은 벡터를 더해줍니다.
- 이 위치 인코딩 벡터는 학습되는 것이 아니라, 사인(sine) 및 코사인(cosine) 함수를 사용하여 미리 정의된 패턴으로 생성됩니다.
- 공식:

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

여기서 pos 는 시퀀스 내 토큰의 위치(0부터 시작), i 는 임베딩 벡터 내 차원의 인덱스, d_{model} .

- 이처럼 고유한 위치 벡터를 임베딩에 더함으로써, 모델은 각 토큰의 절대적 위치뿐만 아니라 상대적 위치까지 파악할 수 있게 됩니다.

5.3.2 멀티 헤드 어텐션 (Multi-Head Attention)

멀티 헤드 어텐션은 트랜스포머의 핵심이자 가장 중요한 구성 요소입니다.

Q, K, V (Query, Key, Value) 개념

트랜스포머의 어텐션 메커니즘은 각 입력 임베딩 H 를 세 가지 다른 역할의 벡터로 변환합니다: 쿼리(Query, Q), 키(Key, K), 값(Value, V).

- 변환 과정:

$$Q = H \cdot W_Q$$

$$K = H \cdot W_K$$

$$V = H \cdot W_V$$

여기서 H 는 입력 임베딩(또는 이전 레이어의 출력), W_Q ,

- 역할:
 - Query (Q): 현재 처리하려는 토큰의 "질문" 또는 "초점"을 나타냅니다. "나는 어

떤 정보를 찾고 있는가?"

- **Key (K)**: 시퀀스 내 다른 모든 토큰의 "키" 또는 "식별자"를 나타냅니다. "이 토큰은 쿼리와 얼마나 관련이 있는가?"
- **Value (V)**: 시퀀스 내 다른 모든 토큰의 "값" 또는 "정보 내용"을 나타냅니다. "쿼리와의 관련성에 따라 이 토큰에서 어떤 정보를 추출해야 하는가?"

Q, K, V 비유: 도서관에서 책 찾기

Q, K, V의 역할을 도서관에서 책을 찾는 과정에 비유하면 이해하기 쉽습니다.

- **Query (Q)**: 당신이 찾고 있는 책의 **제목** (예: "인공지능 개론").
- **Key (K)**: 도서관의 모든 책에 붙어있는 **분류 번호** (예: "004.AI").
- **Value (V)**: 분류 번호에 해당하는 **실제 책의 내용**.

당신은 (Q)제목을 가지고 (K)분류 번호들을 훑어보며 가장 유사한 책을 찾습니다. 유사도가 높은 (K)분류 번호에 해당하는 (V)책의 내용을 가져와 읽는 것입니다.

Q, K, V 예시: 문맥에 따른 단어 의미 조정

"bank account"와 "river bank"라는 두 문맥에서 "bank"라는 단어를 생각해봅시다.

- **기존 임베딩의 한계**: 일반적인 단어 임베딩은 두 문맥에서 "bank"에 동일한 벡터를 할당합니다. 이는 문맥 정보를 반영하지 못합니다.
- **Q, K, V를 통한 문맥 조정**:
 1. "bank"라는 단어의 **Q**를 생성합니다.
 2. 시퀀스 내 다른 단어들("account", "river")의 **K**를 생성합니다.
 3. "bank"의 **Q**와 "account"의 **K**를 비교하여 유사도(어텐션 점수)를 계산합니다. 마찬가지로 "bank"의 **Q**와 "river"의 **K**도 비교합니다.
 4. "bank account" 문맥에서는 "bank"와 "account"의 유사도가 높게 나올 것이고, "river bank" 문맥에서는 "bank"와 "river"의 유사도가 높게 나올 것입니다.
 5. 이 유사도에 따라 각 단어의 **V**를 가중합하여 "bank"의 새로운 표현을 만듭니다. 결과적으로 "bank account"의 "bank"는 금융 관련 정보를, "river bank"의 "bank"는 지형 관련 정보를 더 많이 포함하는 벡터로 조정됩니다.

이처럼 Q, K, V는 모델이 문맥에 따라 단어의 의미를 동적으로 조정하고, 시퀀스 내에서 어떤 단어에 더 집중해야 할지 결정하는 유연성을 제공합니다.

스케일드 닷-프로덕트 어텐션 (Scaled Dot-Product Attention)

Q, K, V를 사용하여 어텐션 출력을 계산하는 핵심 공식은 다음과 같습니다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- QK^T : Q K (
- $/ \sqrt{d_k}$: 계산된 유사도 점수를 키 벡터의 차원 d_k . , .(
- $\text{softmax}(\dots)$: 스케일링된 유사도 점수에 소프트맥스 함수를 적용하여 어텐션 가중

치(확률 분포)를 얻습니다. 이 가중치들은 각 쿼리가 각 값에서 얼마나 많은 정보를 가져와야 하는지를 결정합니다.

- v : 계산된 어텐션 가중치 행렬을 값 행렬 v 에 곱합니다. 이는 각 값 벡터에 해당 가중치를 곱하고 합산하여 최종 어텐션 출력을 생성합니다. 이 출력은 문맥에 따라 가중치가 부여된 정보의 합입니다.

멀티 헤드 (Multi-Head) 어텐션

단일 어텐션 메커니즘만으로는 모델이 다양한 유형의 관계를 동시에 포착하기 어렵습니다. 이를 해결하기 위해 트랜스포머는 멀티 헤드 어텐션(Multi-Head Attention)을 사용합니다.

- 병렬 실행: 여러 개의 "어텐션 헤드"를 병렬로 실행합니다. 각 헤드는 고유한 W_Q, W_K, W_V .
- 다양한 관점 학습: 각 헤드는 입력 시퀀스에서 다른 부분에 집중하거나, 다른 유형의 관계(예: 한 헤드는 구문 관계, 다른 헤드는 의미 관계)를 학습할 수 있습니다.
- 출력 결합: 각 헤드에서 나온 어텐션 출력을 연결(concatenate)한 후, 최종 선형 변환(W_O)을 적용합니다.

5.3.3 잔차 연결 및 정규화 (Residual Connections and Normalization)

트랜스포머는 깊은 신경망의 학습을 안정화하고 성능을 향상시키기 위해 잔차 연결과 레이어 정규화를 사용합니다.

잔차 연결 (Residual Connections, Shortcut Connections)

- 개념: 신경망의 특정 서브 레이어(예: 어텐션 레이어 또는 피드포워드 네트워크)의 입력 x 를 그 서브 레이어의 출력 $\text{Sublayer}(x)$ 에 직접 더하는 방식입니다. 즉, $\text{Output} = x + \text{Sublayer}(x)$ 형태를 가집니다.
- 목적: 깊은 신경망에서 기울기 소실 문제를 완화하고, 학습을 용이하게 합니다. 모델은 $\text{Sublayer}(x)$ 대신 $\text{Sublayer}(x) - x$ (잔차)를 학습하게 되므로, 작은 변화를 학습하는 데 더 효과적입니다.

레이어 정규화 (Layer Normalization)

- 개념: 각 샘플(예: 하나의 문장) 내의 모든 특성(피처)에 대해 정규화를 수행합니다. 이는 배치 정규화(Batch Normalization)와 달리 배치 크기에 독립적입니다.
- 목적: 신경망의 활성화 값 분포를 안정화하여 학습 과정을 가속화하고, 더 큰 학습률을 사용할 수 있게 하여 모델의 수렴 속도와 성능을 향상시킵니다.

5.3.4 마스크드 멀티 헤드 어텐션 (Masked Multi-Head Attention)

디코더에서만 사용되는 특별한 어텐션 메커니즘입니다.

마스크드 멀티 헤드 어텐션의 목적

- 개념: 디코더가 현재 예측하려는 토큰의 미래 정보를 보지 못하도록 가리는(masking) 어텐션 방식입니다.
- 필요성: 텍스트 생성과 같이 순차적으로 다음 토큰을 예측하는 작업(예: "나는 [다음 단어]를 좋아한다"에서 "[다음 단어]"를 예측)에서는 모델이 아직 생성되지 않은 미래 정보를 미리 보아서 안 됩니다. 이는 '정보 유출(data leakage)'로 이어져 모델이 단순히 미래를 베끼는 결과를 초래할 수 있습니다.
- 작동 방식: $Q K^T$, (: -infinity) . 0 , V .

5.4 트랜스포머의 활용: BERT와 GPT

트랜스포머는 현대 자연어 처리(NLP) 분야의 혁명을 이끌었으며, 그 구조를 기반으로 다양한 강력한 모델들이 탄생했습니다. 대표적인 예가 BERT와 GPT입니다.

트랜스포머를 찢어 활용하기

트랜스포머는 대규모 텍스트 데이터로 사전 학습된 후, 특정 작업에 맞게 미세 조정될 수 있습니다. 이 과정에서 트랜스포머의 인코더 또는 디코더 부분을 분리하여 독립적인 모델로 활용할 수 있습니다.

- **BERT (Bidirectional Encoder Representations from Transformers):**
 - 활용 부분: 트랜스포머의 인코더 부분만 사용합니다.
 - 특징: 양방향 문맥 이해에 특화되어 있습니다. 즉, 단어의 의미를 파악할 때 해당 단어의 왼쪽 문맥과 오른쪽 문맥을 모두 동시에 고려합니다.
 - 주요 작업: 텍스트 분류, 개체명 인식, 질의응답, 문장 유사도 측정 등 텍스트 이해(NLU, Natural Language Understanding) 작업에 주로 활용됩니다.
- **GPT (Generative Pre-trained Transformer):**
 - 활용 부분: 트랜스포머의 디코더 부분만 사용합니다 (이때 인코더-디코더 어텐션 부분은 제거되거나 사용되지 않습니다).
 - 특징: 단방향 문맥 이해를 기반으로 다음 토큰을 예측하는 텍스트 생성에 특화되어 있습니다.
 - 주요 작업: 문장 완성, 요약, 대화 생성, 번역 등 텍스트 생성(NLG, Natural Language Generation) 작업에 주로 활용됩니다.
 - 작동 방식: GPT는 질문과 같은 입력을 받으면, 이를 바탕으로 다음 토큰을 순차적으로 예측하여 답변을 생성합니다.

6 트랜스포머의 장점

트랜스포머는 기존 시퀀스 모델의 한계를 극복하고 다음과 같은 강력한 장점들을 제공합니다.

트랜스포머의 주요 장점

- **뛰어난 병렬 처리 능력:** RNN의 순차적 계산 의존성을 제거하고 어텐션 메커니즘을 통해 모든 토큰 간의 관계를 동시에 계산합니다. 이로 인해 GPU와 같은 병렬 컴퓨팅 장치를 최대한 활용하여 학습 속도를 대폭 향상시킬 수 있습니다.
- **효과적인 장거리 의존성 학습:** 어텐션 메커니즘은 시퀀스 내의 모든 토큰에 직접적으로 '주의'를 기울일 수 있으므로, 멀리 떨어진 토큰 간의 복잡한 의존성도 매우 효과적으로 포착하고 학습할 수 있습니다. 이는 RNN의 고질적인 문제였던 장거리 의존성 학습의 어려움을 근본적으로 해결합니다.
- **전이 학습(Transfer Learning)의 용이성:** 트랜스포머는 대규모 데이터로 사전 학습된 후, 특정 하위 작업에 맞게 미세 조정 (fine-tuning)될 수 있습니다. 이러한 전이 학습 패러다임은 다양한 NLP 작업에서 매우 높은 성능을 달성하는 데 기여했습니다.
- **유연한 아키텍처:** 위치 인코딩 덕분에 다양한 길이의 입력 시퀀스에 유연하게 적용될 수 있습니다. 또한, 멀티 헤드 어텐션은 모델이 다양한 관점에서 정보를 처리할 수 있는 유연성을 제공합니다.
- **해석 가능성:** 어텐션 가중치를 분석함으로써 모델이 어떤 입력 토큰에 집중하여 출력을 생성했는지 파악할 수 있어, 모델의 의사결정 과정을 이해하는 데 도움을 줍니다.

7 빠르게 훑어보기 (1페이지 요약)

핵심 개념들을 빠르게 복습할 수 있도록 카드 형태로 요약했습니다.

RNN의 한계

- 기울기 소실: 장거리 의존성 학습 어려움.
- 순차 처리: 병렬화 비효율적.
- 고정 병목: 긴 시퀀스 정보 손실.
- 미래 문맥: 고려 어려움.

어텐션 메커니즘

- 핵심: 각 출력 시점마다 동적 문맥 벡터 생성.
- 작동: $CT = \text{sum}(\alpha * H_j)$.
- 장점: 동적 집중, 장거리 의존성, 가변 길이, 안정성.
- 목표: RNN의 고정 병목 문제 해결.

트랜스포머 등장 배경

- RNN 한계 지속: 어텐션 기반 RNN도 병렬화, 기울기 소실 문제 잔존.
- 철학: "Attention Is All You Need" (RNN 제거).
- 구조: 인코더-디코더.

위치 인코딩

- 문제: 어텐션은 순서 정보 무시.
- 해결: 임베딩에 위치 정보 벡터 추가 (사인/코사인 함수).
- 목적: 토큰의 절대/상대 위치 파악.

Q, K, V (Query, Key, Value)

- Q: 현재 토큰의 '질문' (찾는 정보).
- K: 다른 토큰의 '키' (Q와 관련성 비교).
- V: 다른 토큰의 '값' (추출할 실제 정보).
- 변환: $H * W_Q/K/V$.

스케일드 닷-프로덕트 어텐션

- 공식: $\text{softmax}((Q K^T)/\sqrt{dk}))V$.
- 스케일링: $\sqrt{dk}$ 로 나누어 학습 안정화. 결과: 문맥에 따라 가중치 부여된 정보의 합.

멀티 헤드 어텐션

- 개념: 여러 어텐션 헤드를 병렬 실행.
- 목적: 다양한 관점/관계 동시 학습. 구현: 각 헤드 고유 $W_Q/K/V$.

잔차 연결 및 정규화

- 잔차 연결: $\text{Output} = \text{Input} + \text{Sublayer}(\text{Input})$. 깊은 망 학습 용이.
- 레이어 정규화: 각 샘플 내 특성 정규화. 학습 안정화.

마스크드 어텐션

- 사용처: 디코더에서만.
- 목적: 현재 토큰이 미래 토큰을 보지 못하도록 가림.
- 필요성: 텍스트 생성 시 정보 유출 방지.

트랜스포머의 활용

- BERT: 인코더만 사용. 양방향 문맥 이해 (NLU).
- GPT: 디코더만 사용. 단방향 텍스트 생성 (NLG).

8 자주 묻는 질문 (FAQ)

초심자들이 혼동할 수 있는 질문들을 Q/A 형식으로 정리했습니다.

- **Q: 어텐션 메커니즘과 트랜스포머의 가장 큰 차이점은 무엇인가요?**
 - **A:** 어텐션 메커니즘은 RNN의 성능을 향상시키는 '기술' 또는 '구성 요소'입니다. 반면 트랜스포머는 RNN을 완전히 제거하고 오직 어텐션 메커니즘만으로 시퀀스 모델링을 수행하는 '모델 아키텍처' 자체입니다. 트랜스포머는 어텐션 메커니즘을 핵심으로 사용하지만, RNN을 사용하지 않는다는 점에서 근본적인 차이가 있습니다.
- **Q: Q, K, V를 왜 세 가지로 나누나요? 하나나 두 개로는 안 되나요?**
 - **A:** Q, K, V를 세 가지로 나누는 것은 모델에 더 큰 유연성을 제공하기 위함입니다. Q는 '무엇을 찾고 있는지', K는 '무엇이 있는지', V는 '찾은 것의 내용'을 나타내며, 각각 다른 가중치 행렬을 통해 변환되므로 동일한 입력 토큰이라도 문맥에 따라 다른 역할을 수행할 수 있습니다. 만약 Q, K, V가 모두 동일하다면, 모델은 다양한 관점에서 관계를 학습하기 어렵고 표현력이 제한될 것입니다.
- **Q: 위치 인코딩이 왜 필요한가요? 어텐션이 알아서 순서를 학습할 수는 없나요?**
 - **A:** 어텐션 메커니즘은 본질적으로 입력 토큰들의 순서에 영향을 받지 않습니다(순열 불변성). 즉, "A B C"와 "C B A"를 동일하게 처리할 수 있습니다. 하지만 언어에서는 단어의 순서가 의미를 결정하는 데 매우 중요하므로, 위치 인코딩을 통해 명시적으로 순서 정보를 주입해야 모델이 이를 인지하고 활용할 수 있습니다.
- **Q: 마스크드 어텐션은 언제 사용되며, 왜 필요한가요?**
 - **A:** 마스크드 어텐션은 트랜스포머의 디코더에서 텍스트를 생성할 때 사용됩니다. 모델이 현재 토큰을 예측할 때, 아직 생성되지 않은 미래의 토큰 정보를 보지 못하도록 가리는 역할을 합니다. 이는 모델이 단순히 미래를 '배끼는' 것이 아니라, 현재까지의 정보만을 바탕으로 다음 토큰을 '예측'하도록 강제하여 실제 텍스트 생성 능력을 학습하게 합니다.
- **Q: 트랜스포머는 병렬 처리가 가능하다고 하는데, 왜 학습 비용이 그렇게 비싼가요?**
 - **A:** 트랜스포머는 RNN에 비해 훨씬 효율적인 병렬 처리가 가능하여 학습 속도가 빠릅니다. 하지만 '비용'이 비싸다는 것은 주로 모델의 크기(파라미터 수)와 학습 데이터의 규모 때문입니다. 최신 트랜스포머 모델들은 수십억에서 수천억 개의 파라미터를 가지며, 이를 훈련하기 위해 페타바이트 규모의 텍스트 데이터를 사용합니다. 이 거대한 모델과 데이터를 처리하는 데 막대한 컴퓨팅 자원(GPU 시간)이 소모되므로 비용이 매우 높아집니다.

9 체크리스트

이 강의 내용을 효과적으로 학습하고 이해했는지 스스로 점검해볼 수 있는 체크리스트입니다.

학습 점검 체크리스트

- 퀴즈 복습:
 - CRF가 HMM보다 복잡한 문맥 의존성을 모델링하는 이유를 설명할 수 있는가?
 - LSTM과 CRF 결합이 과거 및 미래 문맥 포착에 어떻게 기여하는지 아는가?
 - 일반 오토인코더의 잠재 공간이 구조화되지 않은 이유와 그 문제점을 설명할 수 있는가?
 - VAE에서 μ 가 입력의 결정론적 함수인 이유를 아는가?
 - VAE의 출력이 비결정론적인 이유(노이즈)를 설명할 수 있는가?
- 어텐션 메커니즘:
 - RNN의 주요 한계점(기울기 소실, 병렬화, 고정 병목)을 나열하고 설명할 수 있는가?
 - 어텐션 메커니즘이 RNN의 한계를 어떻게 극복하는지 설명할 수 있는가?
 - 문맥 벡터(CT), 어텐션 가중치(alpha), 정렬 점수(e)의 개념과 계산 과정을 설명할 수 있는가?
 - 어텐션 메커니즘의 주요 장점(동적 집중, 장거리 의존성, 가변 길이)을 설명할 수 있는가?
- 트랜스포머:
 - 어텐션 기반 RNN 모델이 여전히 가지는 한계점(병렬화, 기울기 소실)을 설명할 수 있는가?
 - 트랜스포머가 "Attention Is All You Need"라는 철학을 가지는 이유를 설명할 수 있는가?
 - 트랜스포머의 인코더-디코더 아키텍처의 주요 구성 요소(멀티 헤드 어텐션, 피드포워드, 잔차 연결, 정규화)를 나열할 수 있는가?
 - 위치 인코딩이 왜 필요하며, 어떻게 작동하는지 설명할 수 있는가?
 - Q(Query), K(Key), V(Value)의 개념과 각각의 역할, 그리고 이들이 어텐션 계산에 어떻게 사용되는지 설명할 수 있는가?
 - 스케일드 닷-프로덕트 어텐션의 공식과 $\sqrt{d_k}$ 로 나누는 이유를 설명할 수 있는가?
 - 멀티 헤드 어텐션이 단일 어텐션보다 유리한 이유를 설명할 수 있는가?
 - 잔차 연결과 레이어 정규화가 트랜스포머에서 어떤 역할을 하는지 설명할 수 있는가?
 - 마스크드 멀티 헤드 어텐션이 디코더에서 사용되는 이유와 그 작동 방식을 설명할 수 있는가?
 - BERT와 GPT가 트랜스포머의 어떤 부분을 활용하며, 각각 어떤 종류의 작업에 특화되어 있는지 설명할 수 있는가?
 - 트랜스포머의 주요 장점(병렬 처리, 장거리 의존성, 전이 학습)을 설명할 수 있는가?